

pyProximation Documentation

Release 1.3.5

Mehdi Ghasemi

Mar 13, 2026

CONTENTS

1 Introduction **3**
- 1.1 Requirements and dependencies . . . 3
- 1.2 Download . . . 3
- 1.3 Installation . . . 3
- 1.4 License . . . 3

2 Measures **5**
- 2.1 Density function . . . 5
- 2.2 Integrals . . . 6
- 2.3 p-norms . . . 6
- 2.4 Drawing samples . . . 7

3 Hilbert Spaces **9**
- 3.1 Orthonormal system of functions . . . 9
- 3.2 OrthSystem . . . 10
- 3.3 Approximation . . . 12
- 3.4 Rational Approximation . . . 14

4 Interpolation **15**
- 4.1 Lagrange interpolation . . . 15
- 4.2 L^2-approximation with discrete measures . . . 16

5 Code Documentation **19**

6 Indices and tables **23**

Python Module Index **25**

Contents:

CHAPTER

ONE

INTRODUCTION

This is a brief documentation for using *pyProximation*. pyProximation provides tools for function approximation in Hilbert spaces. It uses orthogonal systems of functions built via the Gram-Schmidt procedure over measures on arbitrary domains.

1.1 Requirements and dependencies

Since this is a python package, so clearly one needs have python available. pyProximation relies on the following packages:

- **for numerical calculation:**
  - numpy,
  - scipy ,
- **for symbolic computation:**
  - sympy

1.2 Download

pyProximation can be obtained from https://github.com/mghasemi/pyProximation.

1.3 Installation

To install *pyProximation*, run the following in terminal:

```
sudo python setup.py install
```

1.3.1 Documentation

The documentation of *pyProximation* is prepaqred via sphinx.

1.4 License

pyProximation is distributed under MIT license:

1.4.1 MIT License

Copyright (c) 2016 Mehdi Ghasemi

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

CHAPTER

TWO

MEASURES

2.1 Density function

The *pyProximation* implements two different scenarios for measure spaces:

1. Continuous case, where support of the measure is given as a compact subspace (box) of $\mathbb{R}^n$, and
2. Discrete case, where a finite set of points and their weights are given.

2.1.1 Continuous measure spaces

For the continuous case, generally assume that the support of the measure is a product of closed interval, i.e., $\prod_{i=1}^{n}[a_i, b_i]$, where for each i, $a_i < b_i$. Such a set can be defined as a list of ordered pairs of real numbers as `[(a1, b1), (a2, b2), ..., (an, bn)]`. Moreover, when we speak about a subset of the support, we always refer to a box, defined as a list of 2-tuples.

In this case, the measure is given implicitly as a density function $w(x)$. So the measure of a set S is given by

$$\mu(S) = \int_S w(x)dx.$$

For example, the following code defines the Lebesgue measure on $[-1, 1] \times [-1, 1]$ and finds the measure of the set $[0, 1] \times [-1, 0]$:

```
# import the Measure class
from pyProximation import Measure
# define the support of the measure
D = [(-1, 1), (-1, 1)]
# define the measure with the constant density 1
M = Measure(D, 1)
# define a set called S
S = [(0, 1), (-1, 0)]
# find the measure of the set S
print M.measure(S)
```

2.1.2 Discrete measure spaces

In this case, the measure is basically a convex combination of Dirac measure. Given a set $X = \{x_1, \ldots, x_n\}$ and corresponding non-negative weights $w_1, \ldots, w_n$, one defines a measure as $\mu = \sum_{i=1}^{n} w_i \delta_{x_i}$. Then the measure of a subset $S = \{x_{i_1}, \ldots, x_{i_k}\}$ of X is given by

$$\mu(S) = \int_S d\mu = \sum_{j=1}^{k} w_{i_j}$$

The following is a sample code for discrete case:

```
# import the Measure class
from pyProximation import Measure
# define the support and density
D = {'x1':1, 'x2':.5, 'x3':1.1, 'x4':.6}
# define the measure
M = Measure(D)
# define a set called S
S = ['x2', 'x3']
# find the measure of the set S
print M.measure(S)
```

2.2 Integrals

Suppose that a measure space (X, μ) and a measurable function f on X are given. The method `integral` computes $\int_X f d\mu$. If μ is discrete, then f can be a dictionary with keys as points of domain and values as evaluation at each point. Otherwise, f is simply a numerical function:

```
from pyProximation import Measure
from numpy import sqrt
# define the density function
w = lambda x:1./sqrt(1.-x**2)
# define the support
D = [(-1, 1)]
# initiate the measure space
M = Measure(D, w)
# set f(x) = x^2
f = lambda x: x**2
# integrate f(x) w.r.t. w(x)
print M.integral(f)
```

Or in two dimensions:

```
from pyProximation import Measure
from numpy import sqrt
# define the density function
w = lambda x, y:y**2/sqrt(1.-x**2)
# define the support
D = [(-1, 1), (-1, 1)]
# initiate the measure space
M = Measure(D, w)
# set f(x, y) = x^2 + y
f = lambda x, y: x**2 + y
# integrate f(x, y) w.r.t. w(x, y)
print M.integral(f)
```

2.3 *p*-norms

Given a measure space (X, μ) and a measurable function f, the p-norm of f, for a positive p is defined as:

$$\|f\|_p = \left(\int_X |f|^p d\mu \right)^{1/p}.$$

The method `norm(p, f)` calculates the above quantity.

2.4 Drawing samples

Suppose that (X, μ) is a measure space and one wishes to draw a sample of size n from X according to the distribution μ. This can be done by the method `sample(n)` which returns a list of n random points from the support, according to μ.

CHAPTER

THREE

HILBERT SPACES

3.1 Orthonormal system of functions

Let X be a topological space and μ be a finite Borel measure on X. The bilinear function $\langle \cdot, \cdot \rangle$ defined on $L^2(X, \mu)$ as $\langle f, g \rangle = \int_X fg d\mu$ is an inner product which turns $L^2(X, \mu)$ into a Hilbert space.

Let us denote the family of all continuous real valued functions on a non-empty compact space X by $\mathrm{C}(X)$. Suppose that among elements of $\mathrm{C}(X)$, a subfamily A of functions are of particular interest. Suppose that A is a subalgebra of $\mathrm{C}(X)$ containing constants. We say that an element $f \in \mathrm{C}(X)$ can be approximated by elements of A, if for every $\epsilon > 0$, there exists $p \in A$ such that $|f(x) - p(x)| < \epsilon$ for every $x \in X$. The following classical results guarantees when every $f \in \mathrm{C}(X)$ can be approximated by elements of A.

Note

Stone-Weierstrass:

Every element of $\mathrm{C}(X)$ can be approximated by elements of A if and only if for every $x \neq y \in X$, there exists $p \in A$ such that $p(x) \neq p(y)$.

Despite the strong and important implications of the Stone-Weierstrass theorem, it leaves every computational details out and does not give an specific algorithm to produce an estimator for f with elements of A, given an error tolerance ϵ, and the search for a such begins.

Define $\|f\|_\infty$ (the $\sup$ norm of f) of a given function $f \in \mathrm{C}(X)$ by

$$\|f\|_\infty = \sup_{x \in X} |f(x)|,$$

Then the above argument can be read as: For every $f \in \mathrm{C}(X)$ and every $\epsilon > 0$, there exists $p \in A$ such that $\|f - p\|_\infty < \epsilon$.

Let $(V, \langle \cdot, \cdot \rangle)$ be an inner product space with $\|v\|_2 = \langle v, v \rangle^{\frac{1}{2}}$. A basis $\{v_\alpha\}_{\alpha \in I}$ is called an orthonormal basis for V if $\langle v_\alpha, v_\beta \rangle = \delta_{\alpha\beta}$, where $\delta_{\alpha\beta} = 1$ if and only if $\alpha = \beta$ and is equal to *0* otherwise. Every given set of linearly independent vectors can be turned into a set of orthonormal vectors that spans the same sub vector space as the original. The following well-known result gives an algorithm for producing such orthonormal from a set of linearly independent vectors:

Note

Gram–Schmidt

Let $(V, \langle \cdot, \cdot \rangle)$ be an inner product space. Suppose $\{v_i\}_{i=1}^n$ is a set of linearly independent vectors in V. Let

$$u_1 := \frac{v_1}{\|v_1\|_2}$$

and (inductively) let

$$w_k := v_k - \sum_{i=1}^{k-1} \langle v_k, u_i \rangle u_i \text{ and } u_k := \frac{w_k}{\|w_k\|_2}.$$

Then $\{u_i\}_{i=1}^n$ is an orthonormal collection, and for each *k*,

$$span\{u_1, u_2, \cdots, u_k\} = span\{v_1, v_2, \cdots, v_k\}.$$

Note that in the above note, we can even assume that $n = \infty$.

Let $B = \{v_1, v_2, \dots\}$ be an ordered basis for $(V, \langle \cdot, \cdot \rangle)$. For any given vector $w \in V$ and any initial segment of *B*, say $B_n = \{v_1, \dots, v_n\}$, there exists a unique $v \in \text{span}(B_n)$ such that $\|w - v\|_2$ is the minimum:

Note

Let $w \in V$ and *B* a finite orthonormal set of vectors (not necessarily a basis). Then for $v = \sum_{u \in B} \langle u, w \rangle u$

$$\|w - v\|_2 = \min_{z \in \text{span}(B)} \|w - z\|_2.$$

Now, let μ be a finite measure on *X* and for $f, g \in \mathrm{C}(X)$ define $\langle f, g \rangle = \int_X fg d\mu$. This defines an inner product on the space of functions. The norm induced by the inner product is denoted by $\| \cdot \|_2$. It is evident that

$$\|f\|_2 \leq \|f\|_\infty \mu(X), \ \forall f \in \mathrm{C}(X),$$

which implies that any good approximation in $\| \cdot \|_\infty$ gives a good $\| \cdot \|_2$-approximation. But generally, our interest is the other way around. Employing Gram-Schmidt procedure, we can find $\| \cdot \|_2$ within any desired accuracy, but this does not guarantee a good $\| \cdot \|_\infty$-approximation. The situation is favorable in finite dimensional case. Take $B = \{p_1, \dots, p_n\} \subset \mathrm{C}(X)$ and $f \in \mathrm{C}(X)$, then there exists $K_f > 0$ such that for every $g \in \text{span}(B \cup \{f\})$,

$$K_f \|g\|_\infty \leq \|g\|_2 \leq \|g\|_\infty \mu(X).$$

Since *X* is assumed to be compact, $\mathrm{C}(X)$ is separable, i.e., $\mathrm{C}(X)$ admits a countable dimensional dense subvector space (e.g. polynomials for when *X* is a closed, bounded interval). Thus for every $f \in \mathrm{C}(X)$ and every $\epsilon > 0$ one can find a big enough finite *B*, such that the above inequality holds. In other words, good enough $\| \cdot \|_2$-approximations of f give good $\| \cdot \|_\infty$-approximations, as desired.

3.2 OrthSystem

Given a measure space, the `OrthSystem` class implements the described procedure, symbolically. Therefore, it relies on a symbolic environment. Currently, the supported symbolic environment is:

1. *sympy*

3.2.1 Legendre polynomials

Let $d\mu(x) = dx$, the regular Lebesgue measure on $[-1, 1]$ and $B = \{1, x, x^2, \dots, x^n\}$. The orthonormal polynomials constructed from *B* are called *Legendre* polynomials. The nth Legendre polynomial is denoted by $P_n(x)$.

The following code generates Legendre polynomials up to a given order:

```
# the symbolic package
from sympy import *
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 6
# define the measure
D = [(-1, 1)]
M = Measure(D, 1)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.PolyBasis(n)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# print the result
print(S.OrthBase)
```

3.2.2 Chebyshev polynomials

Let $d\mu(x) = \frac{dx}{\sqrt{1-x^2}}$ on $[-1, 1]$ and B as in Legendre polynomials. The orthonormal polynomials associated to this setting are called *Chebyshev* polynomials and the nth one is denoted by $T_n(x)$.

The following code generates Chebyshev polynomials up to a given order:

```
# the symbolic package
from sympy import *
from numpy import sqrt
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 6
# define the measure
D = [(-1, 1)]
w = lambda x: 1./sqrt(1. - x**2)
M = Measure(D, w)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.PolyBasis(n)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# print the result
print(S.OrthBase)
```

3.3 Approximation

Let (X, μ) be a compact Borel measure space and $\mathcal{O} = \{u_1, u_2, \dots\}$ an orthonormal basis of function whose span is dense in $L^2(X, \mu)$. Given a function $f \in L^2(X, \mu)$, then f can be approximated as

$$f = \lim_{n \to \infty} \sum_{i=1}^{n} \langle f, u_i \rangle u_i$$

`OrthSystem.Series` calculates the coefficients $\langle f, u_i \rangle$:

3.3.1 Truncated Fourier series

Let $d\mu(x) = dx$, the regular Lebesgue measure on $[c, c + 2l]$ and $B = \{1, \sin(\pi x), \cos(\pi x), \sin(2\pi x), \cos(2\pi x), \dots, \sin(n\pi x), \cos(n\pi x), \}$. The following code calculates the Fourier series approximation of $f(x) = \sin(x)e^x$:

```
from sympy import *
from numpy import sqrt
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 4
# define the measure
D = [(-1, 1)]
w = lambda x: 1./sqrt(1. - x**2)
M = Measure(D, w)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.FourierBasis(n)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# number of elements in the basis
m = len(S.OrthBase)
# set f(x) = sin(x)e^x
f = sin(x)*exp(x)
# extract the coefficients
Coeffs = S.Series(f)
# form the approximation
f_app = sum([S.OrthBase[i]*Coeffs[i] for i in range(m)])
print(f_app)
```

3.3.2 Fitting Data

Suppose that a set of data points $(x_1, y_1), \dots, (x_m, y_m)$ are given, where $x_i \in \mathbb{R}^n$. To fit a curve or surface to data with minimum average square error, one can employ `OrthSystem` with an arbitrary system of functions to do so. The following example generates random data and fits two surfaces using polynomials and trigonometric functions:

```
from random import uniform
from sympy import Function, Symbol
from sympy.utilities.lambdify import implemented_function, lambdify
# import the module
from pyProximation import Measure, OrthSystem
x = Symbol('x')
y = Symbol('y')
# define dirac function symbolically and numerically
dirac = implemented_function(
    Function('dirac'), lambda a, b: 1 * (abs(a - b) < .000001))
# list of random points
points = [(uniform(0, 1), uniform(0, 1)) for _ in range(50)]
# container for the support of discrete measure
D = {}
# a symbolic function to represent the random points
g = 0
# Random values of the function
vals = []
for p in points:
    t = uniform(-.8, 1)
    vals.append(t)
    g += t * dirac(x, p[0]) * dirac(y, p[1])
    D[p] = 1
# The discrete measure supported at random points
M = Measure(D)
# Orthonormal systems of functions
S1 = OrthSystem([x, y], [(0, 1), (0, 1)])
S1.SetMeasure(M)

S2 = OrthSystem([x, y], [(0, 1), (0, 1)])
S2.SetMeasure(M)
# polynomial basis
B1 = S1.PolyBasis(4)
# trigonometric basis
B2 = S2.FourierBasis(3)
# link the basis to the orthogonal systems
S1.Basis(B1)
S2.Basis(B2)
# form the orthonormal basis
S1.FormBasis()
S2.FormBasis()
# calculate coefficients
cfs1 = S1.Series(g)
cfs2 = S2.Series(g)
# orthogonal approximation
aprx1 = sum([S1.OrthBase[i] * cfs1[i] for i in range(len(S1.OrthBase))])
aprx2 = sum([S2.OrthBase[i] * cfs2[i] for i in range(len(S2.OrthBase))])
# inspect the symbolic approximations
print(aprx1)
print(aprx2)
# compare the first sample numerically
f1 = lambdify((x, y), aprx1, 'numpy')
f2 = lambdify((x, y), aprx2, 'numpy')
```

(continues on next page)

(continued from previous page)

```
print(vals[0], f1(*points[0]), f2(*points[0]))
```

3.4 Rational Approximation

Let $f : X \to \mathbb{R}$ be a continuous function and $\{b_0, b_1, \dots\}$ an orthonormal system of functions over X with respect to a measure μ. Suppose that we want to approximate f with a function of the form $\frac{p(x)}{q(x)}$ where $p = \sum_{j=0}^{m} \alpha_j b_j$ and $q = 1 + \sum_{j=1}^{n} \beta_j b_j$. Let $r(x) = p(x) - q(x)f(x)$ be the residual. We can find the coefficients α_i, β_j such that $\|r\|_2$ be minimum.

Let $L(\alpha, \beta) = r(x) \cdot r(x)$. Then the solution satisfies the following equations:

$$\frac{\partial}{\partial \alpha_i} L = 0,$$
$$\frac{\partial}{\partial \beta_i} L = 0,$$

which is a system of linear equations. This is implemented in `rational.RationalApprox.RatLSQ`:

```
from sympy import Symbol, cos, exp, pi, sin, sinh
from pyProximation import OrthSystem, RationalApprox

x = Symbol('x')
f = sinh(x) * cos(3 * x) + sin(3 * x) * exp(x)
D = [(-pi, pi)]
S = OrthSystem([x], D)
B = S.PolyBasis(5)

# link B to S
S.Basis(B)

# generate the orthonormal basis
S.FormBasis()

# extract the coefficients of approximations
Coeffs = S.Series(f)

# form the approximation
h = sum([S.OrthBase[i] * Coeffs[i] for i in range(len(S.OrthBase))])
T = RationalApprox(S)
g = T.RatLSQ(5, 5, f)

print(h)
print(g)
```

CHAPTER

FOUR

INTERPOLATION

4.1 Lagrange interpolation

Suppose that a list of $m+1$ points $\{(x_0, y_0), \ldots, (x_m, y_m)\}$ in $\mathbb{R}^2$ is given, such that $x_i \neq x_j$, if $i \neq j$. Then

$$p(\mathbf{x}) = \sum_{i=0}^{m} y_i \ell_i(\mathbf{x}),$$

is a polynomial of degree *m* which passes through all the given points. Here

$$\ell_i(\mathbf{x}) = \prod_{j \neq i} \frac{\mathbf{x} - x_j}{x_i - x_j},$$

are *Lagrange Basis Polynomials*.

This procedure can be extended to multivariate case as well.

Suppose that a list of points $\{x_1, \ldots, x_\rho\}$ in $\mathbb{R}^n$ and a list of corresponding values $\{y_1, \ldots, y_\rho\}$ are given. Let's denote by $\mathbf{X}$ the tuple $(\mathbf{X}_1, \ldots, \mathbf{X}_n)$ of variables and $\mathbf{X}_1^{e_1} \cdots \mathbf{X}_n^{e_n}$ by $\mathbf{X}^{\mathbf{e}}$, where $\mathbf{e} = (e_1, \ldots, e_n)$.

If for some $m > 0$, we have $\rho = \binom{m+n}{m}$, then the number of given points matches the number of monomials of degree at most *m* in the polynomial basis. Denote the exponents of these monomials by $\mathbf{e}_i$, $i = 1, \ldots, \rho$ and let

$$D = \begin{pmatrix} x_1^{\mathbf{e}_1} & \ldots & x_1^{\mathbf{e}_\rho} \\ \vdots & & \vdots \\ x_\rho^{\mathbf{e}_1} & \ldots & x_\rho^{\mathbf{e}_\rho} \end{pmatrix}$$

and for $1 \leq j \leq \rho$:

$$D_j = \begin{pmatrix} x_1^{\mathbf{e}_1} & \ldots & x_1^{\mathbf{e}_\rho} \\ \vdots & \vdots & \vdots \\ x_{j-1}^{\mathbf{e}_1} & \ldots & x_{j-1}^{\mathbf{e}_\rho} \\ \mathbf{X}^{\mathbf{e}_1} & \ldots & \mathbf{X}^{\mathbf{e}_\rho} \\ x_{j+1}^{\mathbf{e}_1} & \ldots & x_{j+1}^{\mathbf{e}_\rho} \\ \vdots & \vdots & \vdots \\ x_\rho^{\mathbf{e}_1} & \ldots & x_\rho^{\mathbf{e}_\rho} \end{pmatrix}.$$

Then the polynomial

$$p(\mathbf{X}) = \sum_{i=1}^{\rho} y_i \frac{|D_i|}{|D|},$$

interpolates the given list of points and their corresponding values. Here $|M|$ denotes the determinant of *M*.

The above procedure is implemented in the `Interpolation` module. The following code provides an example in 2 dimensional case:

```
from sympy import *
from pyProximation import Interpolation
# define symbolic variables
x = Symbol('x')
y = Symbol('y')
# initiate the interpolation instance
Inter = Interpolation([x, y], 'sympy')
# list of points
points = [(-1, 1), (-1, 0), (0, 0), (0, 1), (1, 0), (1, -1)]
# corresponding values
values = [-1, 2, 0, 2, 1, 0]
# interpolate
p = Inter.Interpolate(points, values)
# print the result
print(p)
```

4.2 L^2-approximation with discrete measures

Suppose that $\mu = \sum_1^n \delta_{x_i}$ is a measure with n-points in its support. Then the orthogonal system of polynomials consists of at most $n+1$ polynomials. Approximation with these $n+1$ polynomials is essentially same as interpolation:

```
# symbolic variable
x = Symbol('x')
# function to be approximated
g = sin(x)*exp(sin(x)*x)#x*sin(x)
# its numerical equivalent
g_ = lambdify(x, g, 'numpy')
# number of approximation terms
n = 6
# half interval length
l = 3.1
# interpolation points and values
Xs = [[-3], [-2], [-1], [0], [1], [2], [3]]
Ys = [g_(Xs[i][0]) for i in range(7)]
# a discrete measure
supp = {-3:1, -2:1, -1:1, 0:1, 1:1, 2:1, 3:1}
M = Measure(supp)
# orthogonal system
S = OrthSystem([x], [(-l, l)])
# link the measure
S.SetMeasure(M)
# polynomial basis
B = S.PolyBasis(n)
# link the basis to the orthogonal system
S.Basis(B)
# form the orthonormal basis
S.FormBasis()
# calculate coefficients
cfs = S.Series(g)
# orthogonal approximation
aprx = sum([S.OrthBase[i]*cfs[i] for i in range(len(B))])
# interpolate
```

(continues on next page)

(continued from previous page)

```
Intrp = Interpolation([x])
intr = Intrp.Interpolate(Xs, Ys)
print(intr)
```

CHAPTER

FIVE

CODE DOCUMENTATION

class pyProximation.base.**Foundation**

This class contains common features of all modules.

DetSymEnv()

Returns `['sympy']` when sympy is available.

class pyProximation.measure.**Measure**(*dom*, *w=None*)

An instance of this class is a measure on a given set *supp*. The support is either

- a python variable of type *set*, or
- a list of tuples which represents a box in euclidean space.

Initializes a measure object according to the inputs:

- ***dom* must be either**
  - a list of 2-tuples
  - a non-empty dictionary
- ***w* must be a**
  - a function if *dom* defines a region
  - left blank (None) if *dom* is a dictionary

boxCheck(*B*)

Checks the structure of the box *B*. Returns *True* id *B* is a list of 2-tuples, otherwise it returns *False*.

check(*dom*, *w*)

Checks the input types and their consistency, according to the *__init__* arguments.

integral(*f*)

Returns the integral of f with respect to the currwnt measure over the support.

measure(*S*)

Returns the measure of the set *S*. *S* must be a list of 2-tuples.

norm(p, f)

Computes the norm-p of the f with respect to the current measure.

sample(*num*)

Samples from the support according to the measure.

class pyProximation.orthsys.**OrthSystem**(*variables*, *var_range*, *env='sympy'*)

OrthogonalSystem class produces an orthogonal system of functions according to a suggested basis of functions and a given measure supported on a given region.

This basically performs a 'Gram-Schmidt' method to extract the orthogonal basis. The inner product is obtained by integration of the product of functions with respect to the given measure (more accurately, the distribution).

To initiate an instance of this class one should provide a list of symbolic variables *variables* and the range of each variable as a list of lists var_range.

To initiate an orthogonal system of functions, one should provide a list of symbolic variables variables and the range of each these variables as a list of lists var_range.

Basis(*base_set*)

To specify a particular family of function as a basis, one should call this method with a list base_set of linearly independent functions.

FormBasis()

Call this method to generate the orthogonal basis corresponding to the given basis via Basis method. The result will be stored in a property called OrthBase which is a list of function that are orthogonal to each other with respect to the measure measure over the given range Domain.

FourierBasis(*n*)

Generates a Fourier basis from variables consisting of all sin & cos functions with coefficients at most *n*.

PolyBasis(*n*)

Generates a polynomial basis from variables consisting of all monomials of degree at most n.

Series(*f*)

Given a function *f*, this method finds and returns the coefficients of the series that approximates *f* as a linear combination of the elements of the orthogonal basis.

SetMeasure(*M*)

To set the measure which the orthogonal system will be computed, simply call this method with the corresponding distribution as its parameter *dm*; i.e, the parameter is *d(m)* where *m* is the original measure.

SetOrthBase(*base*)

Sets the orthonormal basis to be the given *base*.

TensorPrd(*Bs*)

Takses a list of symbolic bases, each one a list of symbolic expressions and returns the tensor product of them as a list.

inner(*f*, *g*)

Computes the inner product of the two parameters with respect to the measure measure.

project(*f*, *g*)

Finds the projection of f on g with respect to the inner product induced by the measure measure.

pyProximation.rational.**RationalApprox**

alias of *RationalAprox*

class pyProximation.rational.**RationalAprox**(*orth*)

RationalAprox calculates a rational approximation for a given function. RationalApprox is the preferred public alias; this class name is retained for backward compatibility. It takes one argument *orth* which is an instance of OrthSystem and does all the computations in the scope of this object.

RatLSQ(*m*, *n*, *f*)

Calculates a rational function :math:` rac{p}{q}` where p, q consists of the first m, n elements of the orthonormal basis $||f - racpq||_2$ is minimized.

class pyProximation.interpolation.**Interpolation**(*var*, *env='sympy'*)

The Interpolation class provides polynomial interpolation routines in multi variate case.

var is the list of symbolic variables and *env* is the the symbolic tool.

Delta(*idx=-1*)

Construct the matrix corresponding to *idx*'th point, if *idx>0* Otherwise returns the discriminant.

Interpolate(*points*, *vals*)

Takes a list of points *points* and corresponding list of values *vals* and return the interpolant.

Since in multivariate case, there is a constraint on the number of points, it checks for the valididty of the input. In case of failure, describes the type of error occured according to the inputs.

MinNumPoints()

Returns the minimum number of points still required.

Monomials()

Generates the minimal set of monomials for interpolation.

CHAPTER

SIX

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

p

pyProximation.base, 19
pyProximation.interpolation, 21
pyProximation.measure, 19
pyProximation.orthsys, 19
pyProximation.rational, 20